

Lab-01 Report: Generated 2026-09-01 14:08

- **Q1.1):** What happened when the cell was executed? Be specific.
- **A1.1):** It printed the string "Welcome to EE / CprE / CybE 3250!" below inside the cell output
- **Q1.2):** Review the Markdown syntax guide. In the space below, use Markdown to tell me a little bit about yourself. Your response to this question should also include an ordered list, some bolded text, some italicized text, and a link.
- **A1.2):**

Name & Section

Tyler Bibus, Section A

About me

I am a CPRE undergraduate set to graduate *december 2026!*

Currently I TA both CPRE 3810 and CPRE 4870/5870 so I have some background with this content. I will say the setup of this lab with the install script and notebook are very neat.

You can see my portfolio [here!](#) I should maybe make it more professional but I like the cyberpunk aesthetic.

- **Q1.3):** Review the "Short Math Guide for LaTeX", available [here](#). Another resource is the [Overleaf](#) guide to LaTeX. In your response below, produce a reasonably close typesetting of the [Kirchoff integral theorem](#) for a monochromatic wave. Don't spend more than 5 minutes on this part - the objective is not to produce publication-quality typesetting, but rather to demonstrate that you can communicate a mathematical expression clearly and effectively using LaTeX.
- **A1.3):** *note, I used a LaTeX equation editor to make this faster and relearn the syntax. I could not find $\hat{n}$ though :(*

$$U(r) = \frac{1}{4\pi} \int_S \left[U \frac{\partial}{\partial n} \left(\frac{e^{iks}}{s} \right) - \frac{e^{iks}}{s} \frac{\partial U}{\partial n} \right] dS$$

- **Q2.1):** In what ways are these basic arithmetic operators similar to other programming languages you are familiar with, and in what ways are they different? Be specific.
- **A2.1):** Overall the core functionality of the operations is the same, + does addition, * does multiplication, etc. Python is different from something like C though, particularly for division as a & b could be interpreted as integers if you write it in C, and division of integers in C would just produce 2 rather than 2.5 as the type would not be converted to a float/double.

```
import math

mypi = 3 # Good enough for government work

# YOUR CODE STARTS HERE
mypi = 0
i=0
for i in range(100): # I know it says no loops but I cant type out enough
elements to improve it for my own sanity
    mypi += ((-1) ** i) * (1/(2*i + 1))

mypi *= 4
# about 3.1316 is close enough
# YOUR CODE ENDS HERE

relative_error = 100*abs(mypi - math.pi) / math.pi

print(f"Calculated pi as {mypi}, Actual pi as {math.pi}, Relative Error:
{relative_error:.2f}%")
```

Calculated pi as 3.1315929035585537, Actual pi as 3.141592653589793, Relative Error: 0.32%

1 two

- **Q2.2):** Review, but do not execute the following code block. Carefully examine each part and describe what it is doing. Your response should explain the purpose of the variables, functions, operators, control structures, and any other important components of the code. Be as specific as possible; imagine that you are explaining the code to someone who has never seen it before. To prevent accidental execution, a runtime error has been intentionally included on the first line. If you find yourself struggling to analyze the code and would like to test

your understanding, you may temporarily comment out that line and execute the cell, but you should first attempt to reason through the code without running it.

- **A2.2):**

Part A temperature is 85, which is greater than 80 => print out `Hot`

Part B `i` defaults to 0, and loops to 5 => prints out (each on a new line) `0 1 2 3 4`
(I am assuming non-inclusive of 5 but I forgot a bit)

Part C `i` is reinstated as 0, fruit is an array, it would print out => `0 apple 1 banana 1 cherry`

Part D name is Joseph, `.upper` will convert to uppercase and `len` is the length of the string so it would print => `JOSEPH 6`

Part E A is an Integer, b is a Float (maybe a double), c is a Boolean, d is String so
=> `Int Float Boolean String` or something similar

Part F It would print out the average of the grades, printing out the sum of all the scores divided by 4 (which is the number of scores).

Part G It iterates through the array of numbers, adding the current number in numbers to the total, printing out => `10`

Part H iterates from 1 to 6 for `i` and from `i^2` to `i^3` for `j` as a nested for loop. it would print something like `1 1 2 4 2 8` etc

- **Q2.3):** The code examples above contain 3 alternative syntaxes for Python `for` loops, in parts A, B, and F. Describe the similarities and differences between them.
- **A2.3):** I am assuming this means the `python for i in range(somenumbe)` and similar aka in parts B C and G. They all loop over some range. The first one is a very standard for loop iterating from 0 to 4 while the second one with the fruit iterates over the range of the number of elements in the fruit array. The last type of `for` number in numbers iterates over the range of numbers in the numbers array where number is the current number. This is particularly useful for taking each element in an array to make changes to.
- **Q2.4):** Compare the traditional for loop implementation and the list-comprehension implementation shown above. Which approach do you prefer,

and why? There is no single correct answer, so focus on readability, clarity, and ease of understanding when defending your choice.

- **A2.4):** I prefer the first one as it is more clear in what case it is appended but using the second one feels cooler to use.

```
# Print out the final 10 entries in the state_names list

# YOUR CODE STARTS HERE
print(state_names[-10:])
# YOUR CODE ENDS HERE
```

```
['South Dakota', 'Tennessee', 'Texas', 'Utah', 'Vermont', 'Virginia',
'Washington', 'West Virginia', 'Wisconsin', 'Wyoming']
```

- **Q3.1):** In the following code block, apply three other list methods to `pokemon_list`. Briefly describe what each one is doing.
- **A3.1):** See my comments, but I sorted, appended a new pokemon `mew` and popped it back out

```
# Implement three other examples of list methods (e.g., append, remove, sort,
reverse, etc.) and print the results

# YOUR CODE STARTS HERE
pokemon_list.sort() # sorts in ascending order
print(pokemon_list) # print sorted list
pokemon_list.append('mew') # add mew to end of list
print(pokemon_list[-1]) # print new pokemon
popped_pokemon = pokemon_list.pop() # pop pokemon_list
print(popped_pokemon) # print popped pokemon (mew)
# YOUR CODE ENDS HERE
```

```
['bulbasaur', 'charizard', 'garchomp', 'greninja', 'lucario', 'mimikyu',
'pikachu', 'pikachu', 'sylveon', 'umbreon']
mew
mew
```

- **Q3.2):** When we sliced `state_names` above (both times), what was the resulting data type? Be specific.
- **A3.2):** The resulting data type is a string, and a list (of strings)

```
# Create a new list (new_pokemon_list) that includes every other character in
the entries of pokemon_list,
# if that entry contains the letter 'a'

# YOUR CODE STARTS HERE
```

```

print(pokemon_list) # sanity check
new_pokemon_list = []
for pokemon in pokemon_list:
    if 'a' in pokemon:
        new_pokemon_list.append(pokemon[::2])
# YOUR CODE ENDS HERE

print(new_pokemon_list)

```

```

['bulbasaur', 'charizard', 'garchomp', 'greninja', 'lucario', 'mimikyu',
'pikachu', 'pikachu', 'sylveon', 'umbreon']
['blaar', 'caiad', 'grhm', 'geij', 'lcro', 'pkcu', 'pkcu']

```

- **Q3.3):** How is this tuple similar to a list? How is it different? Why might a programmer prefer a tuple when representing an image's dimensions?
- **A3.3):** It is similar to a list as it stores data in an array-like format. They differ as they are immutable, so you cannot edit them. This would be beneficial for representing image dims, as your image size should not be changed particularly if you are doing some operation on it.
- **Q3.4):** In the code block below, both variables contain essentially the same information. Which representation do you find more readable? Which representation would be easier to use in a large software project? Defend your answer.
- **A3.4):** I prefer the dict, it is more readable and provides some more fine control. While it is more work to set up, having the field is helpful, particularly when working with larger datasets. For example in the list the year is not a year, so you would need to document it with a comment or remember (which is bad practice). If it was just the list with no comment I might infer that it could be the GPA instead while the dict is very clear.
- **Q4.1):** What advantage does the second example (that uses function `fahrenheit_to_celsius()`) provide? What is your own personal benchmark for when a block of code deserves to become its own function?
- **A4.1):** It has the advantage of both reducing the amount of code you need to write while centralizing the method. This also means that if you need to change how something is calculated you only need to change it in one spot. My benchmark is if something needs to be done more than once in the same or similar enough way, it needs to be a function (within reason. Clean code has benefits & drawbacks if taken too far).

```

# YOUR CODE STARTS HERE
def moving_average(list, window):
    i=0
    j=window
    return_list = []
    if window > len(list):
        return -1
    while j <= len(list):
        return_list.append(sum(list[i:j])/window)
        i+=1
        j+=1
    return return_list

# YOUR CODE ENDS HERE

myvalues = range(1,25) # A list of values from 1 to 24
print(moving_average(myvalues, 2))
print(moving_average(myvalues, 3))
print(moving_average(myvalues, 4))
print(moving_average(myvalues, 6))
print(moving_average(myvalues, 23))

[1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5, 10.5, 11.5, 12.5, 13.5, 14.5,
15.5, 16.5, 17.5, 18.5, 19.5, 20.5, 21.5, 22.5, 23.5]
[2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0,
16.0, 17.0, 18.0, 19.0, 20.0, 21.0, 22.0, 23.0]
[2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5, 10.5, 11.5, 12.5, 13.5, 14.5, 15.5,
16.5, 17.5, 18.5, 19.5, 20.5, 21.5, 22.5]
[3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5, 10.5, 11.5, 12.5, 13.5, 14.5, 15.5, 16.5,
17.5, 18.5, 19.5, 20.5, 21.5]
[12.0, 13.0]

```

- **Q4.2):** Describe the approach you took to implementing this exercise. Be specific.
- **A4.2):** I first had thought about how I wanted to track the window. I went with using an upper `j = window` and a lower `i = 0` where they both increment by 1. Then I wanted to track a `return_list` where in each window I sum the list from `i` to `j` and then divide by the window size -- appending the result. I might have an off by 1 error here. But I'm pretty sure it is working as expected. I also added an error catch for if the window is larger thinking maybe that was a trick but its not ever actually hit .
- **Q4.3):** Presumably, the programmer of the following code cell expects the value `0.25` to be stored in `epoch_losses` . However, the output may not match their expectations. Explain why.

- **A4.3):** This is because `epoch_losses` is defined globally, but then it is redefined inside the function as a local variable so it is appended inside the function and then discarded in favor of the global version.
- **Q4.4):** If you haven't already, execute the code cell directly above. It doesn't print anything, despite the multiple `print()` statements inside it. Why do you think this is the case?
- **A4.4):** It is because it defined a bunch of functions, including its self, but did not actually execute any of the functions.
- **Q4.5):** Examine the class below before executing it. Will the code run successfully? If not, explain why. In your response, discuss the purpose of `self.name` and why assigning `name = name` inside `__init__()` does not create an attribute that can be accessed later by the object.
- **A4.5):** No, because in the init function it sets a local variable `name = name`, but not the instance variable for the class as `self.name = name`.

```

# YOUR CODE STARTS HERE
class BinaryClassifierStats:
    def __init__(self):
        self.correct = 0
        self.total = 0

    def add_prediction(self, y_true, y_pred):
        self.total += 1
        if y_true == y_pred:
            self.correct += 1

    def accuracy(self):
        if self.total == 0:
            return 0
        else:
            return self.correct / self.total

# YOUR CODE ENDS HERE

stats = BinaryClassifierStats()
print(stats.accuracy())
stats.add_prediction(1, 1)
print(stats.accuracy())
stats.add_prediction(0, 1)
print(stats.accuracy())
stats.add_prediction(1, 1)
print(stats.accuracy())

```

```

0
1.0
0.5
0.6666666666666666

```

- **Q5.1):** While we've encountered the `import module` syntax already in this lab, this is the first time we've seen the *alias* syntax `import module as shortname`. Nearly every Python script you will use in scientific / engineering context will include the line `import numpy as np`. Why do you think that is? In other words, why do we not just see `import numpy`, or even `import numpy as supercalifragilisticexpialidocious`?
- **A5.1):** It makes it simpler to use `as` rather than typing a long name everytime you want to use the package you can just use the shortened name. Importing it with a longer alias defeats this purpose, too.
- **Q5.2):** What does following code block do? Be specific, and try to determine the functionality of `np.arange()` and `np.reshape()`.

- **A5.2):** `arange` does something similar to `range` as it adds elements from the start to end (in this case 0 to 12, 0 inclusive 12 exclusive). The `reshape` breaks this single dimension array into the a multi dim array with 3 rows and 4 cols. its important to note this is also a tuple!!!
- **Q5.3):** What does following code block do? Be specific, and try to determine the functionality of `np.linspace()` .
- **A5.3):** It is creating an array of 100 elements starting at 0 and ending at approx 2π where each step out of 100 is approximately equivalent (each step in this case increments by $2\pi / 100$).
- **Q5.4):** What is the difference between the `@` and the `*` operator in NumPy? Defend your answer using the example in the next code block.
- **A5.4):** the `@` symbol is actually a matrix multiply while `*` is just a normal multiply. The normal multiply does not do a true matrix multiplication which leads to strange results.
- **Q5.5):** Review the NumPy quick start guide. What changes would you make to the preceding code such that updating `bar[0]` would not also update `foo[0]` ? Be specific.
- **A5.5):** I would use some np functions to create a copy of the array. THis can be done with `np.copy()` or `np.array(, copy=True)` or something similar
- **Q5.6):** What information is the following code extracting from `images` ? Be specific.
- **A5.6):** This grabs the shape of `sample_image2`. `sample_image 2` grabs every other patient, every 10th image, all imaging channels, and the first 128 x 128 pixels.
- **Q5.10):** The original array had shape `(2, 3, 4)` , but the filtered array has shape `(8,)` . Why did the shape change, and what does this tell us about how Boolean masking works?
- **A5.10):** The shape changed as the masking was 1D, bringing it down into a 1D list. This tells us that it iterates it element by element and we can use it to bring

it down into a simple list.

- **Q6.1):** The call to `.head()` did not print out the entire DataFrame. Review the Pandas User Guide linked above. You can focus on the section on "Essential Basic Functionality" [link](#). How could you modify this call to print out the entire DataFrame?
- **A6.1):** You can add an argument to the head that is the size of the dataframe or the number of elements you want.

```
# Store the new df as soccer_df2

# YOUR CODE STARTS HERE
soccer_df2 = pd.DataFrame({"Name": soccer_df["Name"], "Country":
soccer_df["Country"]})
# YOUR CODE ENDS HERE

soccer_df2.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 2 columns):
#   Column   Non-Null Count  Dtype
---  -
0    Name     6 non-null      str
1   Country  6 non-null      str
dtypes: str(2)
memory usage: 353.0 bytes
```

- **Q6.2):** Implement one more filter on `soccer_df` on the following code cell. Briefly explain your logic.
- **A6.2):** I want good players, so only ones that have scored 5 or more goals. I masked it by selecting the "Goals" data and masking for `>= 5`

```
# YOUR CODE STARTS HERE
soccer_df[(soccer_df["Goals"] >= 5)]
# YOUR CODE ENDS HERE
```

	Name	Country	Goals
0	Kylian Mbappé	France	7
1	Erling Haaland	Norway	7
2	Lionel Messi	Argentina	7
3	Harry Kane	England	6

- **Q6.3):** Use the following code block to briefly explore `run_df` using Pandas functions. What can you say about the dataset after this initial exploration? Be specific.
- **A6.3):** This dataset is for a run and has the following data: timestamps, latitude, longitude, altitude, distance, speed, heart rate, and cadence. I know there are 2282 entries for all of these datapoints.

```
# Explore run_df using .info(), .describe(), .head(), etc.
```

```
# YOUR CODE STARTS HERE
```

```
run_df.info()
```

```
run_df.tail()
```

```
# YOUR CODE ENDS HERE
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2282 entries, 0 to 2281
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   timestamp        2282 non-null   str
1   lat              2282 non-null   float64
2   lon              2282 non-null   float64
3   altitude_m       2282 non-null   float64
4   distance_m       2282 non-null   float64
5   speed_ms         2282 non-null   float64
6   heart_rate_bpm   2282 non-null   int64
7   cadence_rpm      2282 non-null   int64
dtypes: float64(5), int64(2), str(1)
memory usage: 187.3 KB
```

	timestamp	lat	lon	altitude_m	distance_m	speed_ms	hea
2277	2026-08-15T18:44:40Z	41.575631	-93.693241	271.2	32254.6	2.893	
2278	2026-08-15T18:44:43Z	41.576071	-93.692645	271.0	32276.6	2.939	
2279	2026-08-15T18:44:44Z	41.576154	-93.692529	271.0	32282.3	3.014	
2280	2026-08-15T18:44:48Z	41.576318	-93.692240	271.4	32304.6	3.004	
2281	2026-08-15T18:44:50Z	41.576358	-93.692111	271.4	32314.5	3.331	

```

# Query 1: What was the maximum speed (in mi/hr, not m/s) on the run?
# YOUR CODE STARTS HERE
max_speed = max((run_df["speed_ms"])) * 2.237 # rough conversion to mi/hr from
meter/sec
# YOUR CODE ENDS HERE
print(f"Maximum speed was {max_speed:.2f} mph")

# Query 2: What was the average speed (in mi/hr, not m/s) on the run?
# YOUR CODE STARTS HERE
avg_speed = np.average(run_df["speed_ms"]) * 2.237
# YOUR CODE ENDS HERE
print(f"Average speed was {avg_speed:.2f} mph")

# Query 3: How far was the run (in miles)?
# YOUR CODE STARTS HERE
total_distance = max((run_df["distance_m"])) / 1609 # lazy way to get max
distance
# YOUR CODE ENDS HERE
print(f"Total distance was {total_distance:.2f} miles")

# Query 4: What was the altitude range [max-min] (in meters)?
# YOUR CODE STARTS HERE
alt_range = max(run_df["altitude_m"]) - min(run_df["altitude_m"])
# YOUR CODE ENDS HERE
print(f"Altitude range was {alt_range:.2f} meters")

```

Maximum speed was 7.72 mph
 Average speed was 6.65 mph
 Total distance was 20.08 miles
 Altitude range was 63.40 meters

- **Q6.4):** In the next code cell, use Pandas to evaluate what percentage of the time the heart rate was within 1 standard deviation of the mean heart rate. You can count rows using `len(run_df)` or `run_df.shape[0]` for this purpose, as we

are assuming a constant sampling rate. This will probably require you write more than 1 line of Pandas - below, explain the logic you implemented for this part.

- **A6.4):** I used pandas standard deviation function for the heart rate column. I hope this was not too simple of an implementation as it seems the intention was to use more pd to calculate the standard deviation. after getting the standard deviation I get the average and get the amount within range.

```
# Query 5: Assuming a constant sampling rate, what percentage of the time was
the heart rate within 1 std of the mean?

# YOUR CODE STARTS HERE
total_rows = len(run_df)
print(total_rows)
standard_deviation = run_df["heart_rate_bpm"].std() # pandas provides this
method. I hope this is not incorrect or too simple..
print(standard_deviation)
average_heart_rate = np.average(run_df["heart_rate_bpm"]) # get the average hr
print(average_heart_rate)
# there is probably a more elegant solution, but by getting the amount out of
range we can get in range.
row_out_range = len(run_df[(run_df["heart_rate_bpm"] > (average_heart_rate +
standard_deviation))] + len(run_df[(run_df["heart_rate_bpm"] <
(average_heart_rate - standard_deviation))])
row_in_range = total_rows - row_out_range
print(row_in_range)
percent_time = row_in_range / total_rows * 100 # remember to multiply by
100...
# YOUR CODE ENDS HERE
print(f"Heart rate was within 1 std {percent_time:.2f}% of the time")
```

```
2282
10.888025582228313
157.9290096406661
1589
Heart rate was within 1 std 69.63% of the time
```

- **Q7.1):** Pick any of the examples on the Matplotlib [examples](#) page and replicate it in the code cell below. In your response to this question, explain what the example code is doing, line-by-line. Be specific.
- **A7.1):** A scatter plot like this is unlikely to be useful in the course. See my comments, some of the data parsing for the stock prices is a bit out of my wheelhouse.

```
# YOUR CODE STARTS HERE
# Scatter demo 2
import matplotlib.pyplot as plt #import matplotlib
```

```

import numpy as np # import numpy

import matplotlib.cbook as cbook # cbook is for fetching data

# Load a numpy record array from yahoo csv data with fields date, open, high,
# low, close, volume, adj_close from the mpl-data/sample_data directory. The
# record array stores the date as an np.datetime64 with a day unit ('D') in
# the date column.
price_data = cbook.get_sample_data('goog.npz')['price_data'] #gets a
sampledata
price_data = price_data[-250:] # get the most recent 250 trading days

delta1 = np.diff(price_data["adj_close"]) / price_data["adj_close"][:-1] #
Calculates the price delta across a particular day

# Marker size in units of points^2
volume = (15 * price_data["volume"][:-2] / price_data["volume"][0])**2 #
volume is the amount of stocks sold, this converts it into a normalized marker
size (size of the circles)
close = 0.003 * price_data["close"][:-2] / 0.003 * price_data["open"][:-2] #
color is a normalized type from the close time divided by the open price.

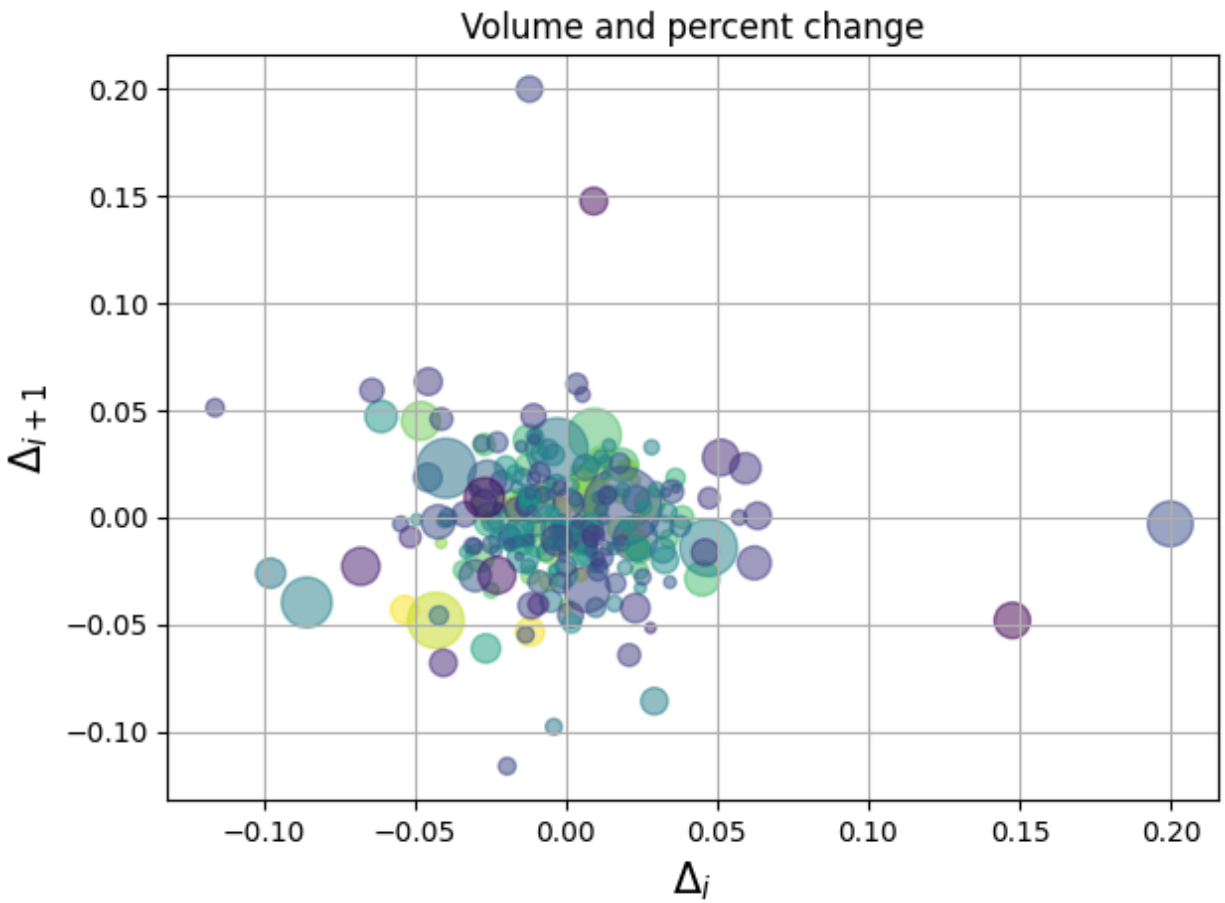
fig, ax = plt.subplots() # create initial figure and axes
ax.scatter(delta1[:-1], delta1[1:], c=close, s=volume, alpha=0.5) # add
scatter plot to axes

ax.set_xlabel(r'$\Delta_i$', fontsize=15) # sets the x axis to Delta_i
ax.set_ylabel(r'$\Delta_{i+1}$', fontsize=15) # sets y axis to Delta_1 + 1
ax.set_title('Volume and percent change')

ax.grid(True) # Adds grid lines
fig.tight_layout() # adjusts the padding around the data, but when
removed/commented out it doesn't appear to do anything noticable

plt.show() # shows the plot (prints it out to cell output in this case)
# YOUR CODE ENDS HERE

```



- **Q7.2):** Create a scatter plot using a scatter plot (using `scatter()`) of the `run_df` data location, with longitude on the x-axis and latitude on the y-axis. Based on your visualization, did the runner complete a full loop during this run? Defend your answer using specific features observed in the plot.
- **A7.2):** Oops, didn't realize the next question (this one), was also scatter. Anyways, after printing it out it looks like a full loop as the end point is pretty close to the starting point.

```
# Scatter plot of lat and long from run_df
# Hint: set c = run_df["distance_m"] to get a color gradient relating to
# distance travelled.

# YOUR CODE STARTS HERE
# pulled from previous code cell.. ignore the comments

longitude = run_df["lon"]
latitude = run_df["lat"]

# Marker size in units of points^2
```

```

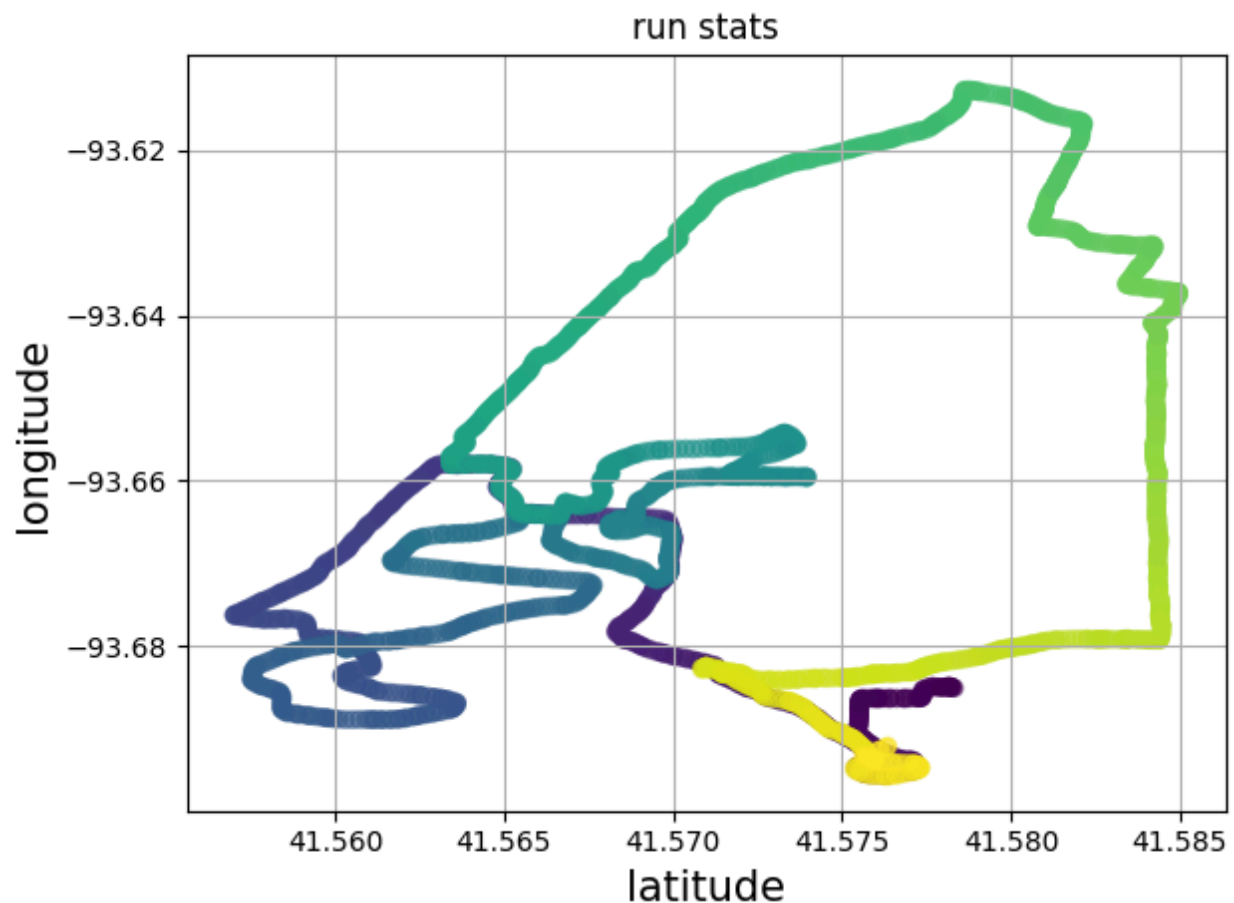
fig, ax = plt.subplots() # create initial figure and axes
ax.scatter(latitude, longitude, c=run_df["distance_m"], alpha=0.5) # add
scatter plot to axes

ax.set_xlabel('latitude', fontsize=15) # sets the x axis to Delta_i
ax.set_ylabel('longitude', fontsize=15) # sets y axis to Delta_1 + 1
ax.set_title('run stats')

ax.grid(True) # Adds grid lines
fig.tight_layout() # adjusts the padding around the data, but when
removed/commented out it doesn't appear to do anything noticeable

plt.show() # shows the plot (prints it out to cell output in this case)
# YOUR CODE ENDS HERE

```



```

# You can set the x-axes to this value and not worry about the elapsed time
calculation
x = range(len(run_df))

# This creates 3 subplots, which you can plot using ax[0], ax[1], and ax[2]
fig, ax = plt.subplots(3, 1, figsize=(12, 8), sharex=True)

# YOUR CODE STARTS HERE
alt_data = run_df["altitude_m"].iloc[::10] # select every 10 to make draw time
not forever

```

```

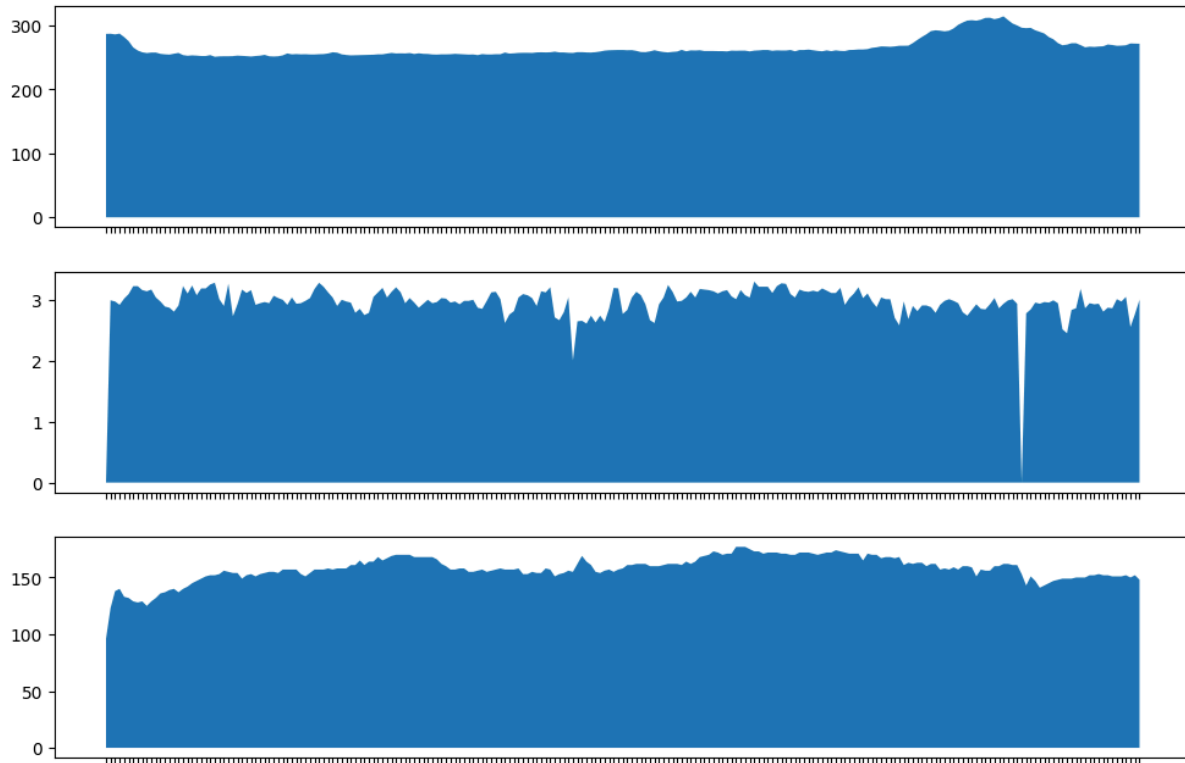
speed_data = run_df["speed_ms"].iloc[:,10]
heart_rate_data = run_df["heart_rate_bpm"].iloc[:,10]
timestamp_data = run_df["timestamp"].iloc[:,10]

ax[0].fill_between(x=timestamp_data, y1=alt_data, y2=0)
ax[1].fill_between(x=timestamp_data, y1=speed_data, y2=0)
ax[2].fill_between(x=timestamp_data, y1=heart_rate_data, y2=0) # X axis is not
readable but its fine for this case I think

# YOUR CODE ENDS HERE

```

<matplotlib.collections.FillBetweenPolyCollection at 0x7fa3370ae600>



- **Q8.1):** Before executing the code below, take a moment to recall the differentiation rules from Calculus I and determine the derivative of $\frac{dy}{dx}$ for $y(x) = x^3$
- **A8.1):** I believe it is $3x^2$
- **Q8.2):** The value of `x.grad` (the gradient of y with respect to x) evaluated to `12.0`. Explain why, and be specific.
- **A8.2):** Its because the derivative of y is $3x^2$ and x is $2 = 3 * 2^2 = 12$. The gradient is the derivative with respect to x which in this case is 2.

- **Q8.3):** When we print out `y4`, we see that it includes both the tensor's numeric value, but also `grad_fn=<SinBackward0>`. What does this suggest about how PyTorch is keeping track of the operations used to create `y4`? Why might remembering that `y4` was generated using the `torch.sin` function be useful when computing derivatives?
- **A8.3):** They need to know the special function (i.e. Sin, Add, etc) as tracking it allows them to more easily keep track of the functions used as different functions have different derivatives.
- **Q8.4):** In the following code cell, use PyTorch to perform another non-trivial computation. Call `print_computation_graph()` to generate a new computational graph. Explain how the graph captures the sequence of operations needed for PyTorch to later compute gradients using `backward()`.
- **A8.4):** The graph appears to recursively build in with the order of operations needed for gradients by tracking all of the equations that will be used to compute the gradient.

```
# YOUR CODE STARTS HERE
p = torch.tensor(6.7, requires_grad=True)

n = p**5 + torch.exp(p) - p**p + torch.cos(p)
n.backward()
print(p.grad)
# YOUR CODE ENDS HERE

print_computation_graph(n.grad_fn)
```

```
tensor(-983176.6875)
AddBackward0
  SubBackward0
    AddBackward0
      PowBackward0
        AccumulateGrad
      ExpBackward0
        AccumulateGrad
    PowBackward1
      AccumulateGrad
      AccumulateGrad
  CosBackward0
    AccumulateGrad
```

- **Q8.5):** Let $y = \sin(\log(x^2 + 1))$. In the next code cell, plot this function along with its $\frac{dy}{dx}$ over the range `[-8, 8]`. Do not use any direct knowledge of $\frac{dy}{dx}$, instead use automatic differentiation to get the result. In the space below, explain your code, and focus on your `.plot()` commands.
- **A8.5):** First we build our function y as $\sin(\log(x^2 + 1))$ using torch for the functions. Then we apply `sum()` before `backward()` to reduce to a scalar and compute the gradients. Then we can get the data via the `detach()` and `numpy()` functions to convert to a numpy array. We then also can get the `grad` dot function to get the gradient of x for the other figure.

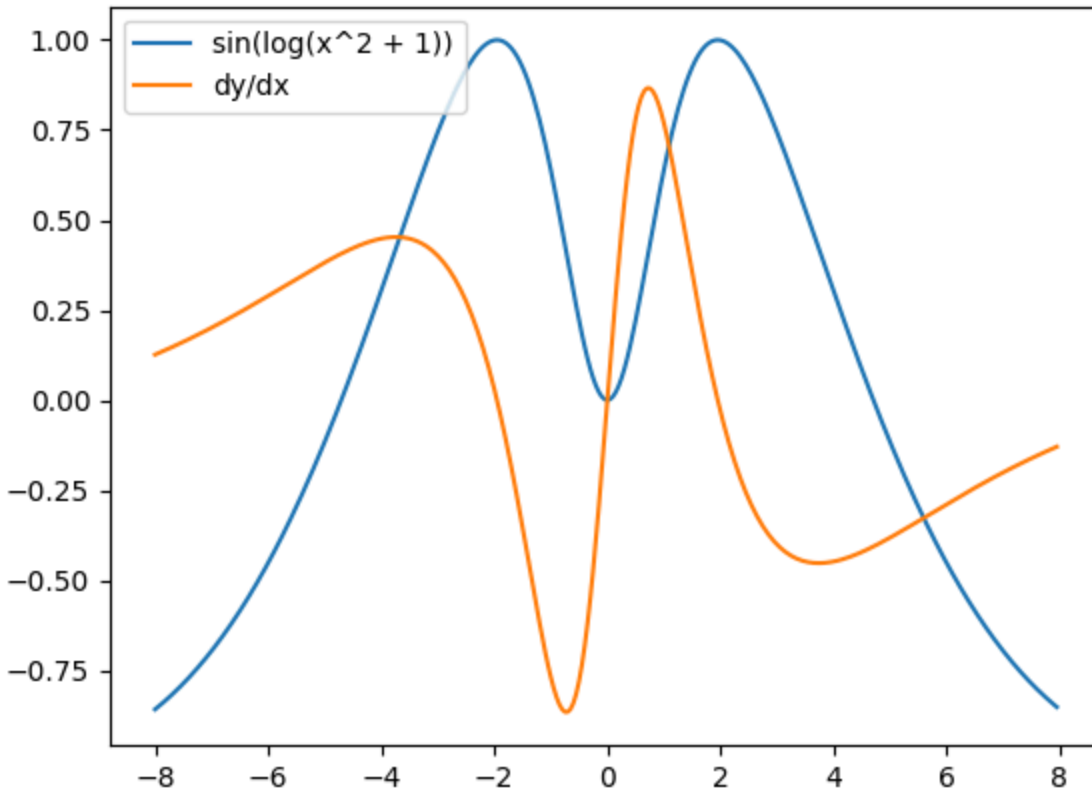
```
# x in the range [-8, 8)
x = torch.arange(-8, 8, 0.05)
x.requires_grad = True

# Hint: if you call y.sum() you will generate a scalar from your otherwise
vector computation.
# sum() does not change the partial derivatives with respect to x_1, x_2,
etc., like we saw for the dot product.

# Hint #2: matplotlib doesn't like tensors, but you can convert back to NumPy
array before plotting

# YOUR CODE STARTS HERE
y = torch.sin(torch.log(x**2 + 1))
y.sum().backward()
dx_data = x.grad.detach().numpy()
x_data_numpy = x.detach().numpy()
y_data_numpy = y.detach().numpy()
#print(x_data_numpy)

plt.figure()
plt.plot(x_data_numpy, y_data_numpy, label="sin(log(x^2 + 1))")
plt.plot(x_data_numpy, dx_data, label="dy/dx")
plt.legend()
plt.show()
# YOUR CODE ENDS HERE
```



- **Q9.1):** Execute the following four code cells and review the error messages they generate. Explain, in plain English (do not just copy/paste), the nature of each error.
- **A9.1):** 1: foobar is not a defined function/variable, if you defined it as "3" it would print "3". 2: values[5] is out of range as values only has 3 elements. 3: The error is because you are defining "number" as a string and trying to add to it, which you cannot add an int to a string. 4: meen is not a recognized variable and it looks like it is just a mis spelling
- **Q9.2):** In the following code cell, execute `help()` on at least three functions that are in either NumPy, Pandas, Matplotlib, or PyTorch. What is something that you have learned through this exercise?
- **A9.2):** for the numpy arange function I figured out that you can actually change the order and change the buffer. For the CPP module of torch I figured out that the it has a lot of advanced and scary functions, but has nn as a submodule so its the parent of nn. Finally I looked at ReLU which applies the rectified linear unit function on each element, this is a non-linear function that is just $\max(0, x)$.

- **Q9.3):** To what extent did you use AI on this Lab-01 assignment? (Remember, it is allowed, if not exactly celebrated). Did you run into any of the issues discussed above?
- **A9.3):** I had AI explain to me the first part of torch with the gradients. I was just having a good deal of confusion of where the 12 was coming from and then I realized that the print statements were printing something else. For that particular part I was just using an in-browser AI that was explaining `.grad` and `.backward()` rather than an integrated harness. The other thing I did was to review how to convert a tensor to numpy, which in my TensorFlow experience was also `numpy()` but not also `detach`. (maybe the `detach` wasn't necessary).